



Q1 Student Grade Book Application

10 marks

You are building a student grade book application. Write a Python program that:

- (a) Stores each student's name and marks in a dictionary (key = name, value = list of 5 subject marks).
- (b) Computes total, average, and grade (O: ≥ 90 , A+: ≥ 80 , A: ≥ 70 , B+: ≥ 60 , B: ≥ 50 , F: < 50) for each student.
- (c) Finds the class topper (highest average) and the subject with the highest overall mean.
- (d) Displays a formatted result table sorted by average in descending order.

Use dictionary comprehension and list operations. Minimum 5 students.

Write complete, syntactically correct Python programs using lists, tuples, dictionaries, and sets with proper indentation.

Your Code

```
1 students = {
2     "Arun": [85, 90, 78, 92, 88],
3     "Bhavya": [70, 65, 80, 75, 60],
4     "Charan": [95, 92, 89, 94, 97],
5     "Divya": [55, 60, 45, 50, 40],
6     "Esha": [88, 79, 91, 85, 90]
7 }
8 def grade(avg):
9     return "O" if avg>=90 else "A+" if avg>=80 else "A" if avg>=70 else "B" if avg>=60 else "C" if avg>=50 else "F"
10 result = {n: (sum(m), round(sum(m)/5,2), grade(sum(m)/5)) for n, m in students.items()}
11 topper = max(result.items(), key=lambda x: x[1][1])
12
13 subj_mean = [round(sum(m[i] for m in students.values())/len(students),2) for i in range(5)]
14 best_subj = subj_mean.index(max(subj_mean)) + 1
15
16 print(f"{'Name':<10}{'Total':<8}{'Avg':<8}{'Grade':<6}")
17 for n, (t, a, q) in sorted(result.items(), key=lambda x: x[1][1], reverse=True):
```

Input

Output

Name	Total	Avg	Grade
Charan	467	93.4	O
Arun	433	86.6	A+
Esha	433	86.6	A+
Bhavya	350	70.0	A
Divya	250	50.0	F

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Alidena Tharun Kumar Reddy ✓
192572068RD

Your Code

```
1 accounts = {}
2
3 def create_account(acc_no, name, balance):
4     accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
5
6 def deposit(acc_no, amount):
7     if acc_no in accounts:
8         accounts[acc_no]['balance'] += amount
9         accounts[acc_no]['txns'].append(('Deposit', amount))
10
11 def withdraw(acc_no, amount):
12     if acc_no in accounts and accounts[acc_no]['balance'] >= amount:
13         accounts[acc_no]['balance'] -= amount
14         accounts[acc_no]['txns'].append(('Withdraw', amount))
15     else:
16         print('Insufficient balance')
17
18 def mini_statement(acc_no):
19     for t in accounts[acc_no]['txns'][-5:]:
20         print(t)
21
22 create_account(1001, 'Ravi', 10000)
23 deposit(1001, 5000)
24 withdraw(1001, 20000)
25 withdraw(1001, 3000)
26 mini_statement(1001)
27 print('Balance:', accounts[1001]['balance'])
```

Input

stdin input

Output

```
Insufficient balance
('Deposit', 5000)
('Withdraw', 3000)
Balance: 12000
```




Q2 Debugging and Rewriting

10 marks

The program below simulates a simple banking system using lists and dictionaries. It has FOUR bugs. Identify each bug with reason and rewrite the correct program.

```
accounts = {}  
def create_account(acc_no, name, balance):  
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}  
def deposit(acc_no, amount):  
    if acc_no in accounts:  
        accounts[acc_no]['balance'] =+ amount  
        accounts[acc_no]['txns'].append(('Deposit', amount))  
def withdraw(acc_no, amount):  
    if accounts[acc_no]['balance'] > amount:  
        accounts[acc_no]['balance'] -= amount  
        accounts[acc_no]['txns'].append(('Withdraw', amount))  
    else:  
        print('Insufficient balance')  
def mini_statement(acc_no):  
    for t in accounts[acc_no]['txns'][-5]:  
        print(t)  
create_account(1001, 'Ravi', 10000)  
deposit(1001, 5000)  
withdraw(1001, 20000)  
withdraw(1001, 3000)  
mini_statement(1001)  
print('Balance:', accounts[1001]['balance'])
```

For debugging questions: clearly state the error, the line number, the reason, and the corrected line.